

第九組

網路小說推薦系統

資工2B 412410085	張子頡
資工2B 412410291	林鈺丞
資工2B 412410929	呂秉諺
資工2B 412411281	劉奕霈
資工2B 412411315	張庭瑄
資工2B 412411810	金庭聿



目錄

01 前言

02 系統分析與設計

03 資料庫邏輯設計

04 實體建置

05 測試

06 結論

07 工作分配

08 參考資料



前言

動機

大部分小說平台的推薦書籍功能多半與讀者曾閱讀的書或者收藏的書無關聯
希望能推薦讀者「可能感興趣」的書籍

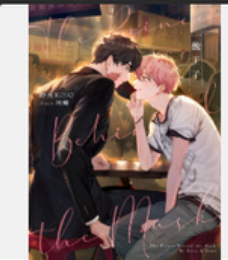
前言

未登入

小說：查無此人，請勿再寄 類別：懸疑推理

已登入

更多感興趣



醜王子
BL耽美
家世顯赫、才識出色、身材挺拔， 聞若澄具備人生勝利組該有的一切條件， 除了他那張…



叛逆玩家
BL耽美
◆都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突…



無瞳之眼03
BL耽美
提線木偶抗拒著束縛正義小警察×神秘大作家——高潮迭起的懸疑篇章疑雲再起……



星降：北域凰城
奇幻科幻
她有許許多的身份，只是目前是不著調的賞金法師。一直以來，她都不願意接受有關…



竹先生
GL百合
曾經有個人說她很愛我。現在的社會或許開放，但是非主流的戀情總會有人說閒話，她…

更多感興趣



醜王子
BL耽美
家世顯赫、才識出色、身材挺拔， 聞若澄具備人生勝利組該有的一切條件， 除了他那張…



叛逆玩家
BL耽美
◆都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突…



無瞳之眼03
BL耽美
提線木偶抗拒著束縛正義小警察×神秘大作家——高潮迭起的懸疑篇章疑雲再起……

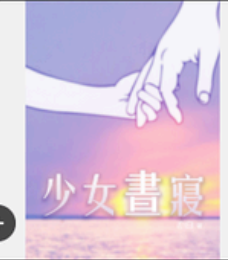


殘愛
GL百合
全天下都知道，蠟祝恩跟瑪希雅相看兩厭，如果某天，她們卻被迫一起出任務怎麼辦？…



一遇唐琰誤終身
愛情
渣男令她負債累累，更是步步緊逼，她滿心怨恨，盯上了小三的哥哥，那個豪門公子哥…

更多感興趣



少女書癡
GL百合
在公司保險櫃前一細一細取出鈔票的情景，被少女用手機拍了下來。「姊姊，你這樣不…



關閉太陽
奇幻科幻
血肉果汁機人氣歌曲改編小說・2023《看你聽的》音樂X漫畫X小說跨域特企作品因為…



窒愛
BL耽美
這個夜很冷，細雨打在街邊那個失了魂的漂亮男人身上，令小高忍不住多看了幾眼。他…



福氣囡少女
輕鬆搞笑
因為哥哥離家出走，她不得不女扮男裝進入男校就讀，卻發現學校裡充滿各種謎團……



一遇唐琰誤終身
愛情
渣男令她負債累累，更是步步緊逼，她滿心怨恨，盯上了小三的哥哥，那個豪門公子哥…

更多感興趣



星降：北域凰城
奇幻科幻
她有許許多的身份，只是目前是不著調的賞金法師。一直以來，她都不願意接受有關…



窒愛
BL耽美
這個夜很冷，細雨打在街邊那個失了魂的漂亮男人身上，令小高忍不住多看了幾眼。他…



我在愛情的盡頭看見了你
愛情
老王樂隊人氣歌曲改編小說・2023《看你聽的》音樂X漫畫X小說跨域特企作品所有的…



餘城涼
愛情
天帝四子青澀君初遇妖族餘涼，本來只是她妖族的身份勾起了他的興趣，卻不想，接…



幽洲願
此生唯求你
愛情
趙十六第一次見忘生時，他被水鬼纏住，她救了他。他講究知恩圖報，從此點上了她，…

前言

未登入

小說：查無此人，請勿再寄 類別：懸疑推理

已登入

更多感興趣



醜王子
BL耽美
家世顯赫、才識出色、身材挺拔， 倘若澄具備人生勝利組該有的一切條件， 除了他那張…



叛逆玩家
BL耽美
◆都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突…



無瞳之眼03
BL耽美
提線木偶抗拒著束縛正義小警察×神秘大作家——高潮迭起的懸疑篇章疑雲再起……



星降：北域凰城
奇幻科幻
她有許許多多的身份，只是目前是不著調的賞金法師。一直以來，她都不願意接受有關…



竹先生
GL百合
曾經有個人說她很愛我。現在的社會或許開放，但是非主流的戀情總會有人說閒話，她…

更多感興趣



醜王子
BL耽美
家世顯赫、才識出色、身材挺拔， 倘若澄具備人生勝利組該有的一切條件， 除了他那張…



叛逆玩家
BL耽美
◆都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突…



無瞳之眼03
BL耽美
提線木偶抗拒著束縛正義小警察×神秘大作家——高潮迭起的懸疑篇章疑雲再起……



殘愛
GL百合
全天下都知道，蠟祝恩跟瑪希雅相看兩厭，如果某天，她們卻被迫一起出任務怎麼辦？…



一遇唐琰誤終身
愛情
渣男令她負債累累，更是步步緊逼，她滿心怨恨，盯上了小三的哥哥，那個豪門公子哥…

更多感興趣



少女書癡
GL百合
在公司保險櫃前一細一細取出鈔票的情景，被少女用手機拍了下來。「姊姊，你這樣不…



關閉太陽
奇幻科幻
血肉果汁機人氣歌曲改編小說・2023《看你聽的》音樂X漫畫X小說跨域特企作品因為…



室愛
BL耽美
這個夜很冷，細雨打在街邊那個失了魂的漂亮男人身上，令小高忍不住多看了幾眼。他…



福氣囡少女
輕鬆搞笑
因為哥哥離家出走，她不得不女扮男裝進入男校就讀，卻發現學校裡充滿各種謎團……

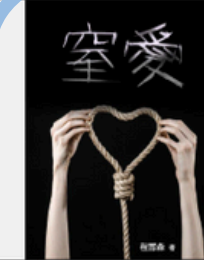


一遇唐琰誤終身
愛情
渣男令她負債累累，更是步步緊逼，她滿心怨恨，盯上了小三的哥哥，那個豪門公子哥…

更多感興趣



星降：北域凰城
奇幻科幻
她有許許多多的身份，只是目前是不著調的賞金法師。一直以來，她都不願意接受有關…



室愛
BL耽美
這個夜很冷，細雨打在街邊那個失了魂的漂亮男人身上，令小高忍不住多看了幾眼。他…



我在愛情的盡頭看見了你
愛情
老王樂隊人氣歌曲改編小說・2023《看你聽的》音樂X漫畫X小說跨域特企作品所有的…



餘城涼
愛情
天帝四子青澀君初遇妖族餘涼，本來只是她妖族的身份勾起了他的興趣，卻不想，接…



幽洲願：此生唯求你
愛情
趙十六第一次見忘生時，他被水鬼纏住，她救了他。他講究知恩圖報，從此點上了她，…

前言

未登入

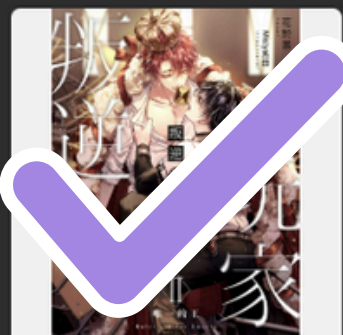
小說：查無此人，請勿再寄 類別：懸疑推理

已登入



前言

你可能會喜歡



叛逆玩家

BL耽美

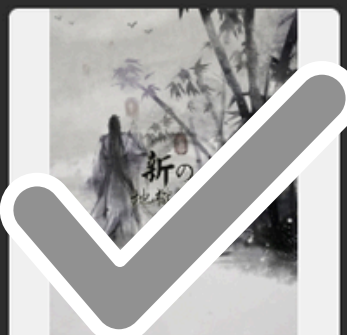
♣都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突…



婚非得已：隱婚老公牆上

愛情

稀里糊塗睡了個牛郎，丟下五百塊不夠，再加點就是了，有必要滿城通緝，尋人啟事滿…



新的地獄遊記

靈異驚悚

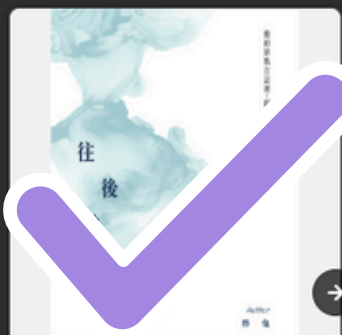
一本絕對與宗教無關的警世小說。這是一個，無論你信不信，它都真實存在的空間；…



吃定嫡妃，腹黑王爺謀天

愛情

她是21世紀女警，執行任務時一個不小心掉下大樓，成了大樑王朝丞相的廢柴嫡女，卻…



往後餘生

BL耽美

褚雲走了。留下同居人鄭安書孤身一人。他們結婚，但他們不是情人。他們同居，但他…

你可能會喜歡



重生後將陰鬱老公寵上天

愛情

【重生+現代言情+豪門總裁+寵妻+甜寵+腹黑+女強+追夫】 重生前，鹿綾有最好的…



【都市/懸疑/罪案/心理】

懸疑推理

一直以成為像父親一樣的正直法官為人生目標的柏皓霖發現父親當年被害一案另有真兇…



霸凌者的告別式

文學寫實

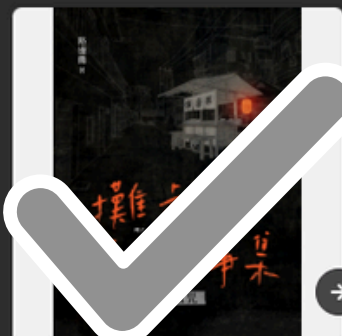
雙胞胎妹妹世理自殺了，穿著她最喜歡的水手制服家人自殺的衝擊、告別式上的惡作劇…



我的笨蛋妹妹

青春校園

十六歲的妹妹因為遭受霸凌成為家裡蹲，又因為不明原因，心智年齡倒退至國小程度。…



攤叔叔短篇故事集：-1組

靈異驚悚

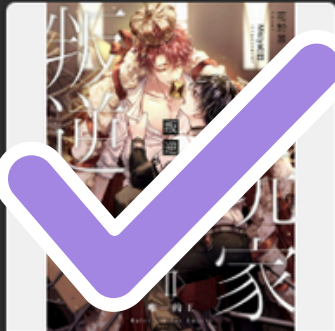
我在住宅區的小巷子裡開了一家咖啡店，某天晚上，一個意料之外的客人來到店裡，是…

前言

你可能會喜歡

2/10

5/10



叛逆玩家
BL耽美

♣都市傳說瘋狗攻 × 盛世美顏反差受 如果王位只能有一座，那必須是我的。林慕突...



婚非得已：隱婚老公牆上寵
宋卿の言

稀里糊塗睡了個牛郎，丟下五百塊不夠，再加點就是了，有必要滿城通緝，尋人啟事滿...



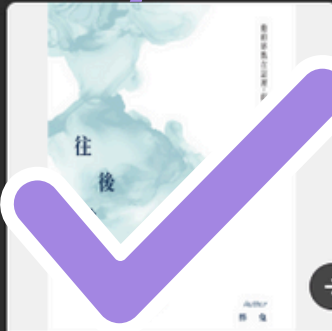
新的地獄遊記
靈異驚悚

一本絕對與宗教無關的警世小說。這是一個，無論你信不信，它都真實存在的空間；...



吃定嫡妃，腹黑王爺謀天
靈異驚悚

她是21世紀女警，執行任務時一個不小心掉下大樓，成了大樑王朝丞相的廢柴嫡女，卻...



往後餘生
BL耽美

褚雲走了。留下同居人鄭安書孤身一人。他們結婚，但他們不是情人。他們同居，但他...

你可能會喜歡

2/10



重生後將陰鬱老公寵上天
靈異驚悚

【重生+現代言情+豪門總裁+寵妻+甜寵+腹黑+女強+追夫】 重生前，鹿綾有最好的...



【都市/懸疑/罪案/心理】懸疑推理
靈異驚悚

一直以成為像父親一樣的正直法官為人生目標的柏皓霖發現父親當年被害一案另有真兇...



霸凌者的告別式
文學寫實

雙胞胎妹妹世理自殺了，穿著她最喜歡的水手制服家人自殺的衝擊、告別式上的惡作劇...



我的笨蛋妹妹
青春校園

十六歲的妹妹因為遭受霸凌成為家裡蹲，又因為不明原因，心智年齡倒退至國小程度。...



攤叔叔短篇故事集：-1組
靈異驚悚

我在住宅區的小巷子裡開了一家咖啡店，某天晚上，一個意料之外的客人來到店裡，是...

前言

目的

- 合理有效的推薦系統
- 模擬mojoin網頁
- 結合演算法與資料庫來推薦小說

系統分析與設計

資料收集

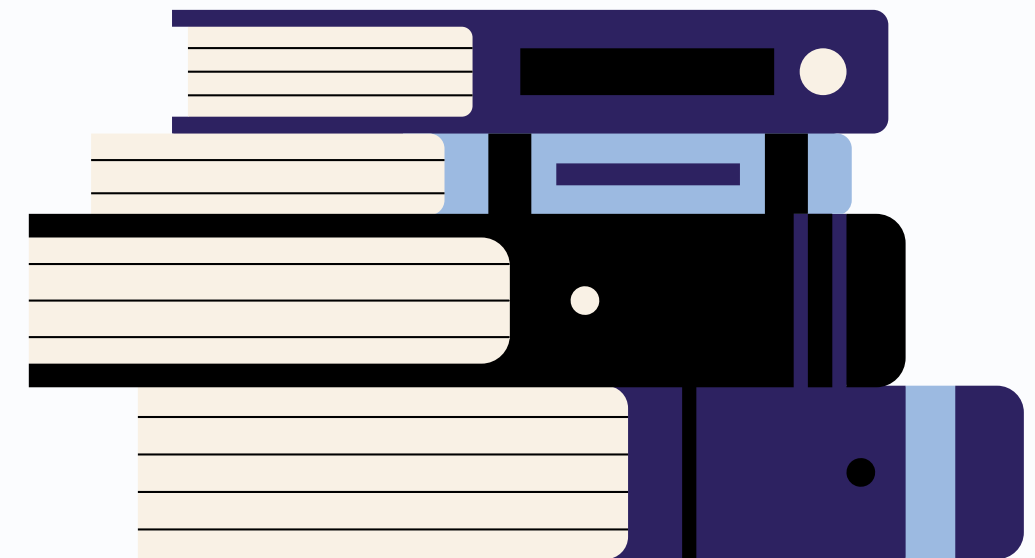
人工統計「Mojoin線上漫畫、小說平台」中的「小說」部分直至民國114年2月28日以前之書籍內容，並建立Excel表格以利後續使用。

系統設計工具

- 資料庫開發工具：MySQL
- 系統設計工具：Flask、Python、HTML、CSS

使用對象

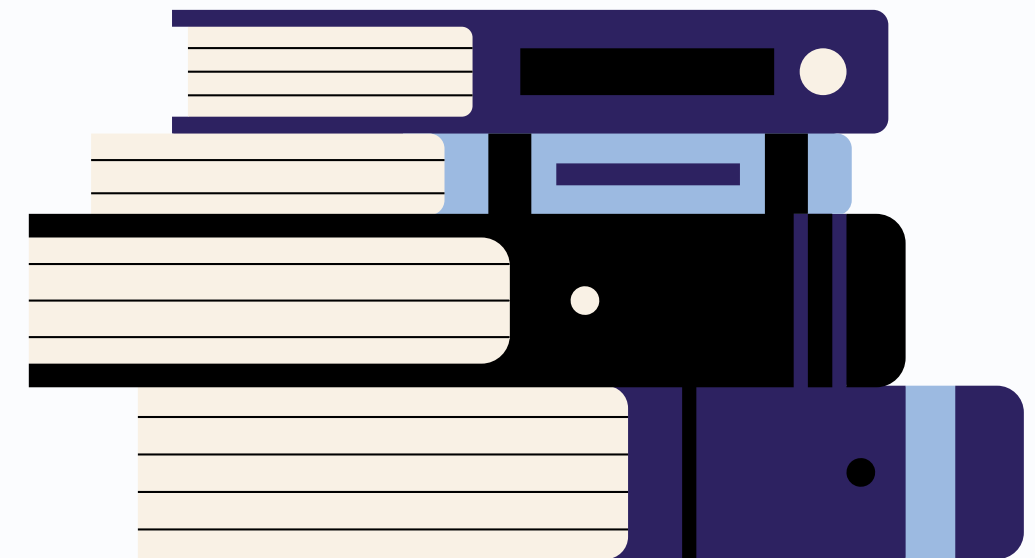
- 一般用戶：享有較符合個人喜好的推薦內容
- 網站管理者：小說登錄



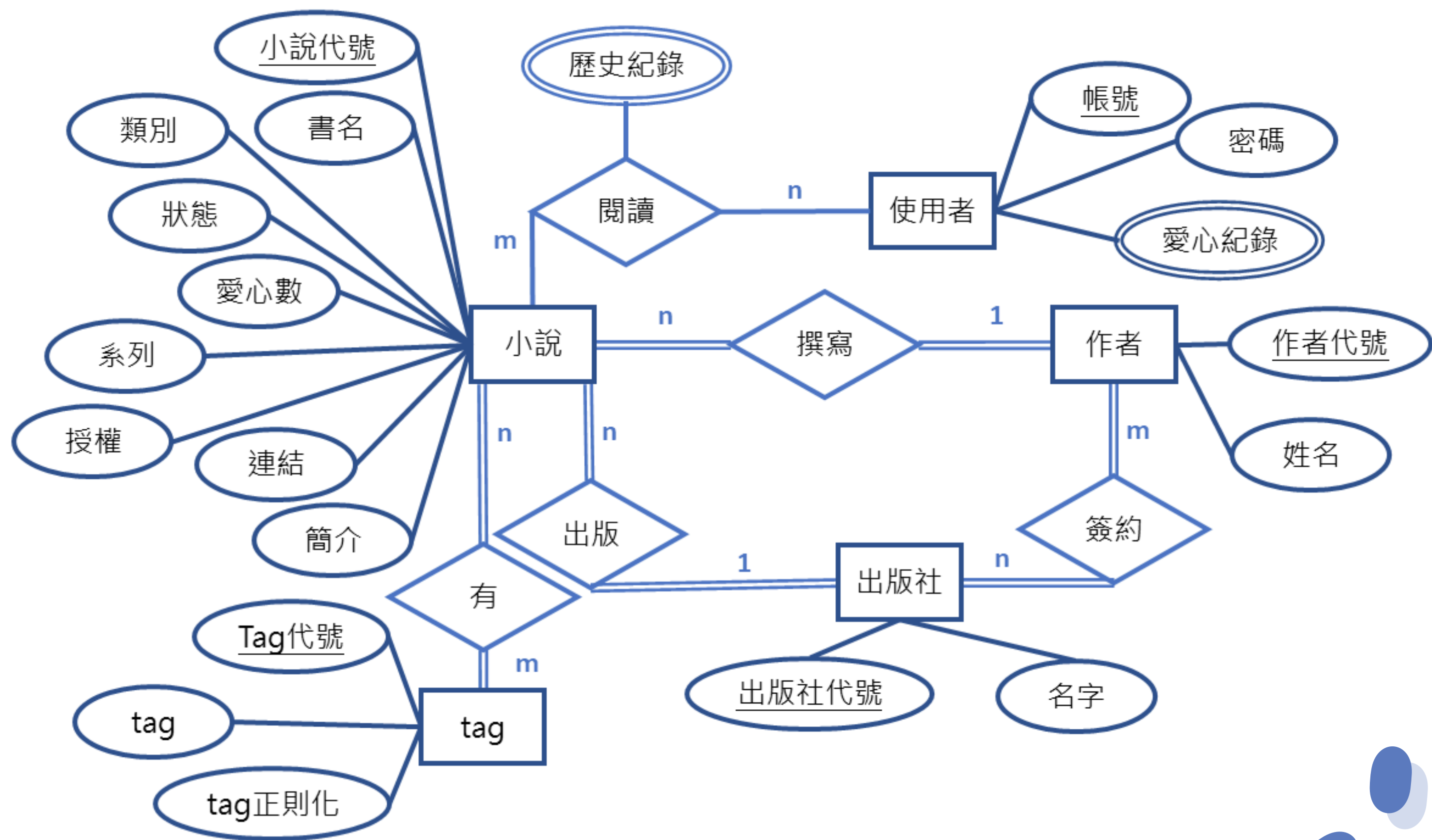
系統分析與設計

系統需求

- 所有書籍均需登錄於該系統內
- 可瀏覽網頁中所有書籍的資訊
- 可經由類型、出版社、作者篩選書籍
- 可經由作者姓名、書籍名稱、出版社名稱或關鍵字查找書籍
- 管理者可任意增加或删除書籍
- 使用者須登入方可使用平台之推薦書籍功能
- 系統可記錄使用者閱讀紀錄、收藏紀錄
- 系統可綜合使用者的喜好推薦其可能感興趣之書籍
- 系統可依據使用者當前查看的書推薦其可能感興趣之書籍



資料庫邏輯設計 E-R 關係圖



資料庫邏輯設計 E-R圖轉換關聯表

- novel(nIndex, Title, Category, State, Love, Series, SeriesN, Warrant, Link,

Introduction, aIndex, pIndex)

- author (aIndex, Name)

- publish (pIndex, Name)

- tag (tIndex, Tag, TagNor)

- user (Account, Password)

- noveltag(nIndex, tIndex)

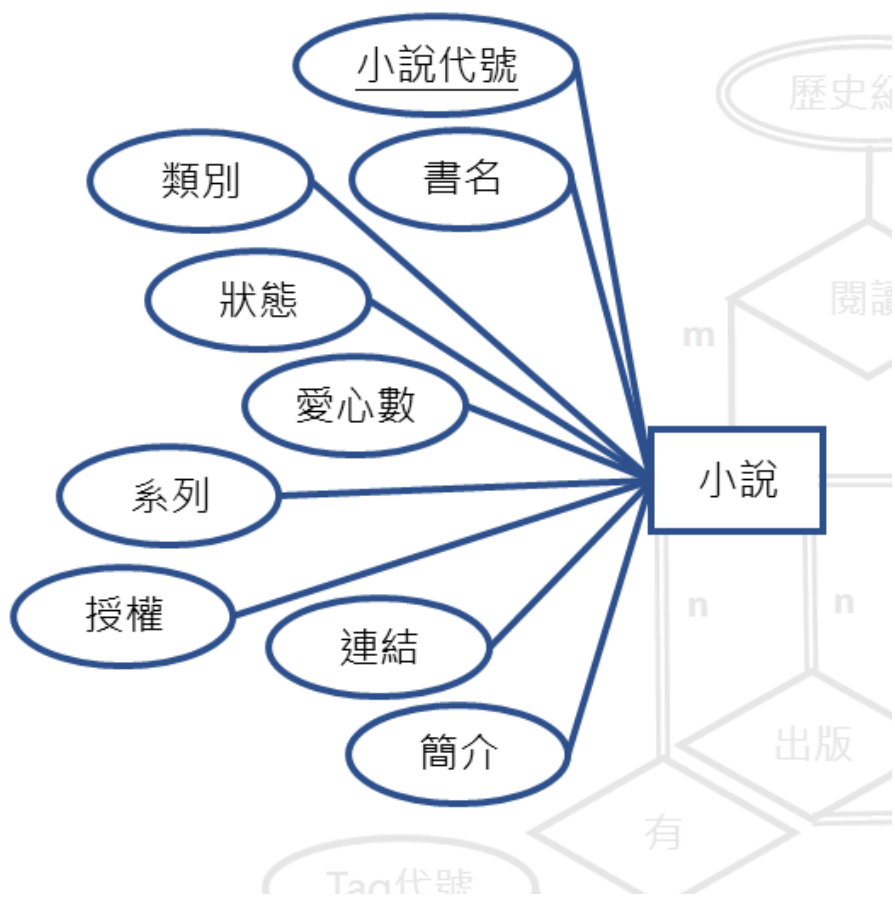
- sign(pIndex, aIndex)

- read(Account, nIndex, History)

- userlove(Account, LoveRecord)

關聯表各欄位屬性規劃

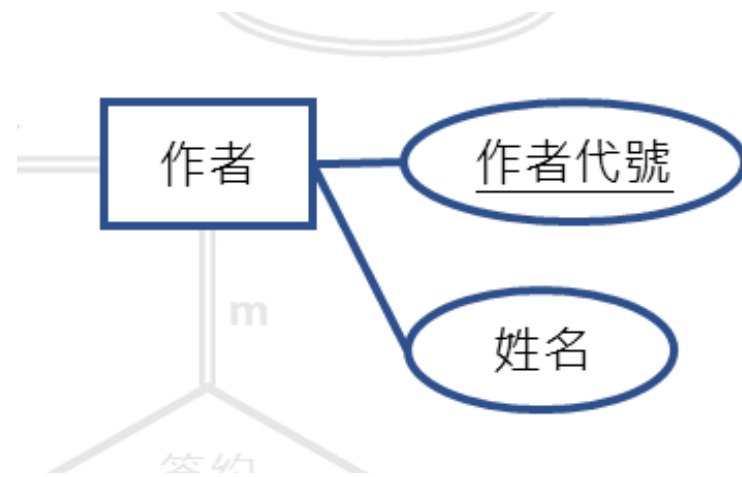
novel



<u>小說代號</u>	書名	類別	狀態
<u>nIndex</u>	Title	Category	State
int	varchar(50)	char(5)	char(2)
愛心數	系列(1)	系列(2)	授權
Love	Series	SeriesN	Warrant
int	Int	int	char(15)
連結	介紹	作者代號	出版商代號
Link	Introduction	<u>aIndex</u>	<u>pIndex</u>
varchar(40)	text	Int	int

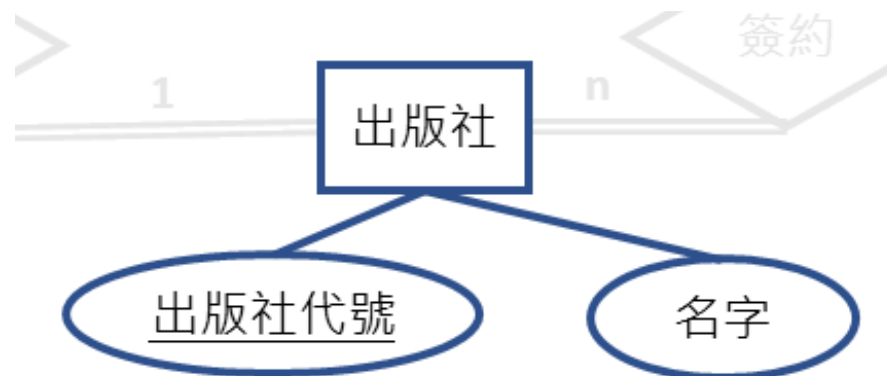
關聯表各欄位屬性規劃

author



<u>作者代號</u>	姓名
<u>aIndex</u>	Name
int	char(20)

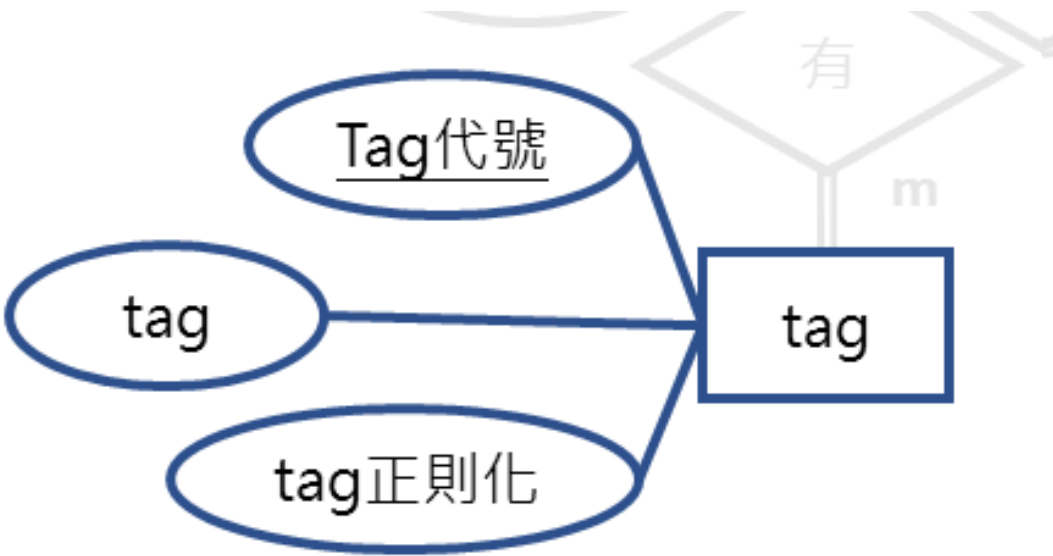
publish



<u>出版社代號</u>	姓名
<u>pIndex</u>	Name
int	char(15)

關聯表各欄位屬性規劃

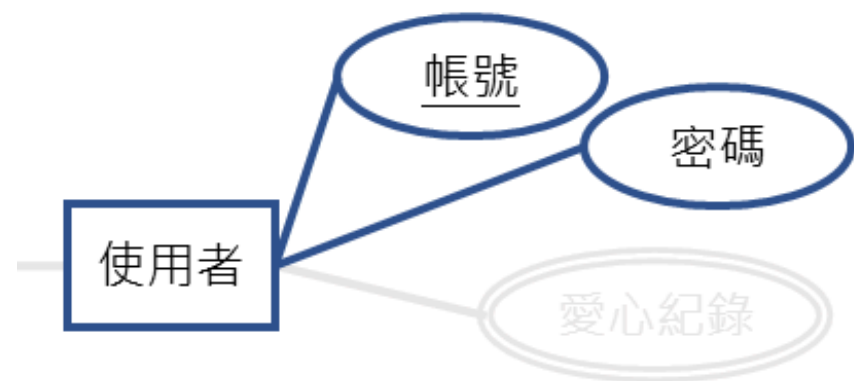
tag



<u>tag代號</u>	tag	tag正則化
<u>tIndex</u>	Tag	TagNOR
int	char(25)	char(25)

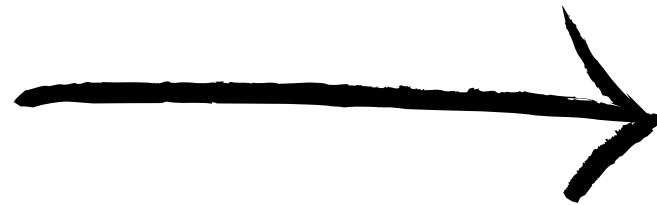
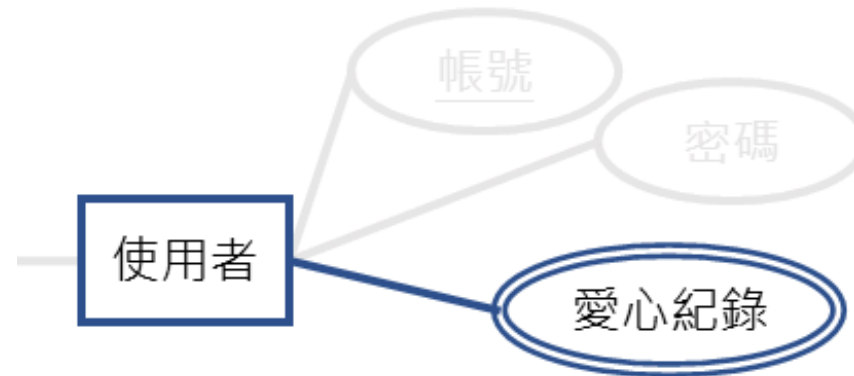
關聯表各欄位屬性規劃

user



<u>帳號</u>	密碼
<u>Account</u>	Password
char(20)	char(20)

userlove



<u>帳號</u>	愛心紀錄
<u>Account</u>	<u>LoveRecord</u>
char(20)	int

關聯表各欄位屬性規劃

noveltag

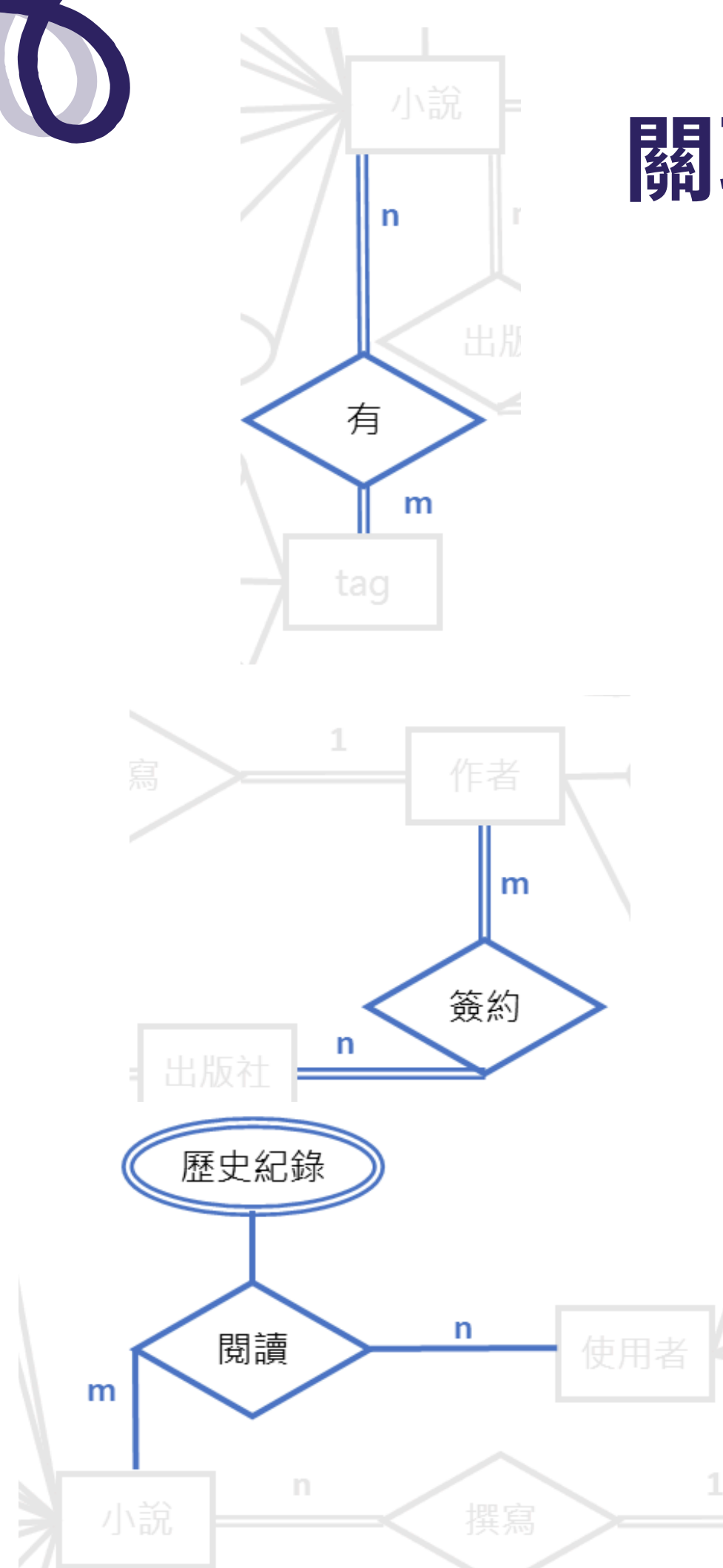
<u>小說代號</u>	<u>Tag代號</u>
<u>nIndex</u>	<u>tIndex</u>
int	int

sign

<u>出版社代號</u>	<u>作者代號</u>
<u>pIndex</u>	<u>aIndex</u>
int	int

read

<u>帳號</u>	<u>小說代號</u>	歷史紀錄
<u>Account</u>	<u>nIndex</u>	History
char(20)	int	int





關聯表各欄位屬性規劃

正規化分析

關聯表中的每一個屬性其值皆為Atomic Value：

符合 novel的「系列」為**複合屬性**，原先為「系列編號・書籍集數」(例：7.8)，因而拆解為「系列(1)」與「系列(2)」

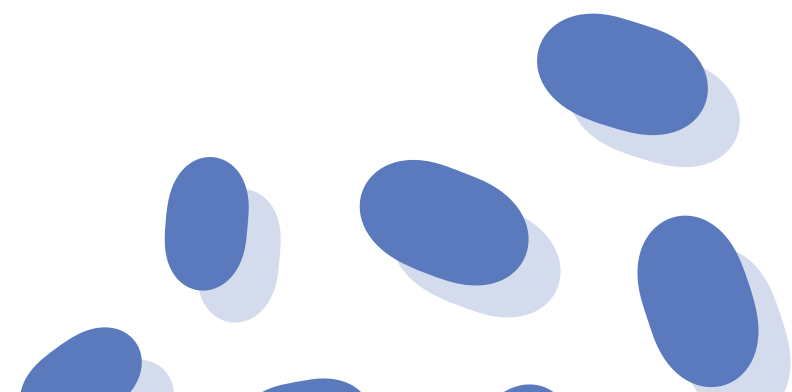
1NF

2NF

關聯表中所有非鍵值屬性皆為完全功能相依於主鍵：**符合**

3NF

關聯表中不存在非鍵值屬性遞移相依於主鍵：**符合**



實體建置 SQL命令

建立table (table依字母排序)

author

```
CREATE TABLE `author` (  
  `aIndex` int unsigned NOT NULL,  
  `Name` char(20) DEFAULT NULL,  
  PRIMARY KEY (`aIndex`),  
  UNIQUE KEY `idAuthor_UNIQUE` (`aIndex`)  
) ENGINE=InnoDB  
DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_0900_ai_ci
```

novel

```
CREATE TABLE `novel` (  
  `nIndex` int unsigned NOT NULL,  
  `Title` varchar(50) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  `Category` char(5) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  `State` char(2) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  `Love` int unsigned NOT NULL,  
  `Series` int DEFAULT NULL,  
  `SeriesN` int DEFAULT NULL,  
  `Warrant` char(15) DEFAULT NULL,  
  `Link` varchar(40) NOT NULL,  
  `Introduction` text CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  `aIndex` int unsigned DEFAULT NULL,  
  `pIndex` int unsigned DEFAULT NULL,  
  PRIMARY KEY (`nIndex`),  
  UNIQUE KEY `nIndex_UNIQUE` (`nIndex`),  
  KEY `fk_novel_publish` (`pIndex`),  
  KEY `fk_novel_author` (`aIndex`),  
  CONSTRAINT `fk_novel_author` FOREIGN KEY (`aIndex`)  
    REFERENCES `author` (`aIndex`) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT `fk_novel_publish` FOREIGN KEY (`pIndex`)  
    REFERENCES `publish` (`pIndex`) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

實體建置 SQL命令

建立table (table依字母排序)

noveltag

```
CREATE TABLE `noveltag` (  
  `nIndex` int unsigned NOT NULL,  
  `tIndex` int unsigned NOT NULL,  
  PRIMARY KEY (`nIndex`,`tIndex`),  
  KEY `tIndex_idx` (`tIndex`),  
  CONSTRAINT `fk_noveltag_novel` FOREIGN KEY (`nIndex`)  
    REFERENCES `novel` (`nIndex`) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT `fk_noveltag_tag` FOREIGN KEY (`tIndex`)  
    REFERENCES `tag` (`tIndex`) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

publish

```
CREATE TABLE `publish` (  
  `pIndex` int unsigned NOT NULL,  
  `Name` char(15) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci NOT NULL,  
  PRIMARY KEY (`pIndex`),  
  UNIQUE KEY `pIndex_UNIQUE` (`pIndex`)  
) ENGINE=InnoDB DEFAULT  
  CHARSET=utf8mb4  
  COLLATE=utf8mb4_0900_ai_ci
```

實體建置 SQL命令

建立table (table依字母排序)

read

```
CREATE TABLE `read` (  
  `Account` char(20) NOT NULL,  
  `nIndex` int NOT NULL,  
  `History` int unsigned NOT NULL,  
  PRIMARY KEY (`Account`,`nIndex`),  
  CONSTRAINT `Account` FOREIGN KEY (`Account`)  
    REFERENCES `user` (`Account`) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

sign

```
CREATE TABLE `sign` (  
  `pIndex` int unsigned NOT NULL,  
  `aIndex` int unsigned NOT NULL,  
  PRIMARY KEY (`pIndex`,`aIndex`),  
  KEY `aIndex_idx` (`aIndex`),  
  CONSTRAINT `aIndex` FOREIGN KEY (`aIndex`)  
    REFERENCES `author` (`aIndex`) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT `pIndex` FOREIGN KEY (`pIndex`)  
    REFERENCES `publish` (`pIndex`) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

實體建置 SQL命令

建立table (table依字母排序)

tag

```
CREATE TABLE `tag` (  
  `tIndex` int unsigned NOT NULL,  
  `Tag` char(25) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,  
  `TagNOR` char(25) CHARACTER SET utf8mb4  
    COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,  
  PRIMARY KEY (`tIndex`),  
  UNIQUE KEY `tIndex_UNIQUE` (`tIndex`)  
) ENGINE=InnoDB  
  DEFAULT CHARSET=utf8mb4  
  COLLATE=utf8mb4_0900_ai_ci
```

user

```
CREATE TABLE `user` (  
  `Account` char(20) NOT NULL,  
  `Password` char(20) NOT NULL,  
  PRIMARY KEY (`Account`),  
  UNIQUE KEY `Account_UNIQUE` (`Account`)  
) ENGINE=InnoDB  
  DEFAULT CHARSET=utf8mb4  
  COLLATE=utf8mb4_0900_ai_ci
```

userlove

```
CREATE TABLE `userlove` (  
  `Account` char(20) NOT NULL,  
  `LoveRecord` int unsigned NOT NULL,  
  PRIMARY KEY (`Account`,`LoveRecord`),  
  KEY `fk_LoveRecord_idx` (`LoveRecord`),  
  CONSTRAINT `fk_Account` FOREIGN KEY (`Account`)  
    REFERENCES `user` (`Account`) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT `fk_LoveRecord` FOREIGN KEY (`LoveRecord`)  
    REFERENCES `novel` (`nIndex`) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

實體建置 SQL命令

匯入資料(以csv檔儲存並直接匯入)

```
mysql> USE mojoin;
Database changed
mysql> LOAD DATA LOCAL INFILE
-> 'D:/Users/peggy/Github/databaseproject-G9/Schema/csv/(table name).csv'
-> INTO TABLE `(table name)`
-> CHARACTER SET utf8mb4
-> FIELDS TERMINATED BY ',' -- 依據檔案的分隔符號
-> LINES TERMINATED BY '\n' -- 依據檔案每列的分隔方式
-> IGNORE 1 LINES;
```

*table name為各自的table名稱以及csv檔名稱

實體建置 搜尋演算法

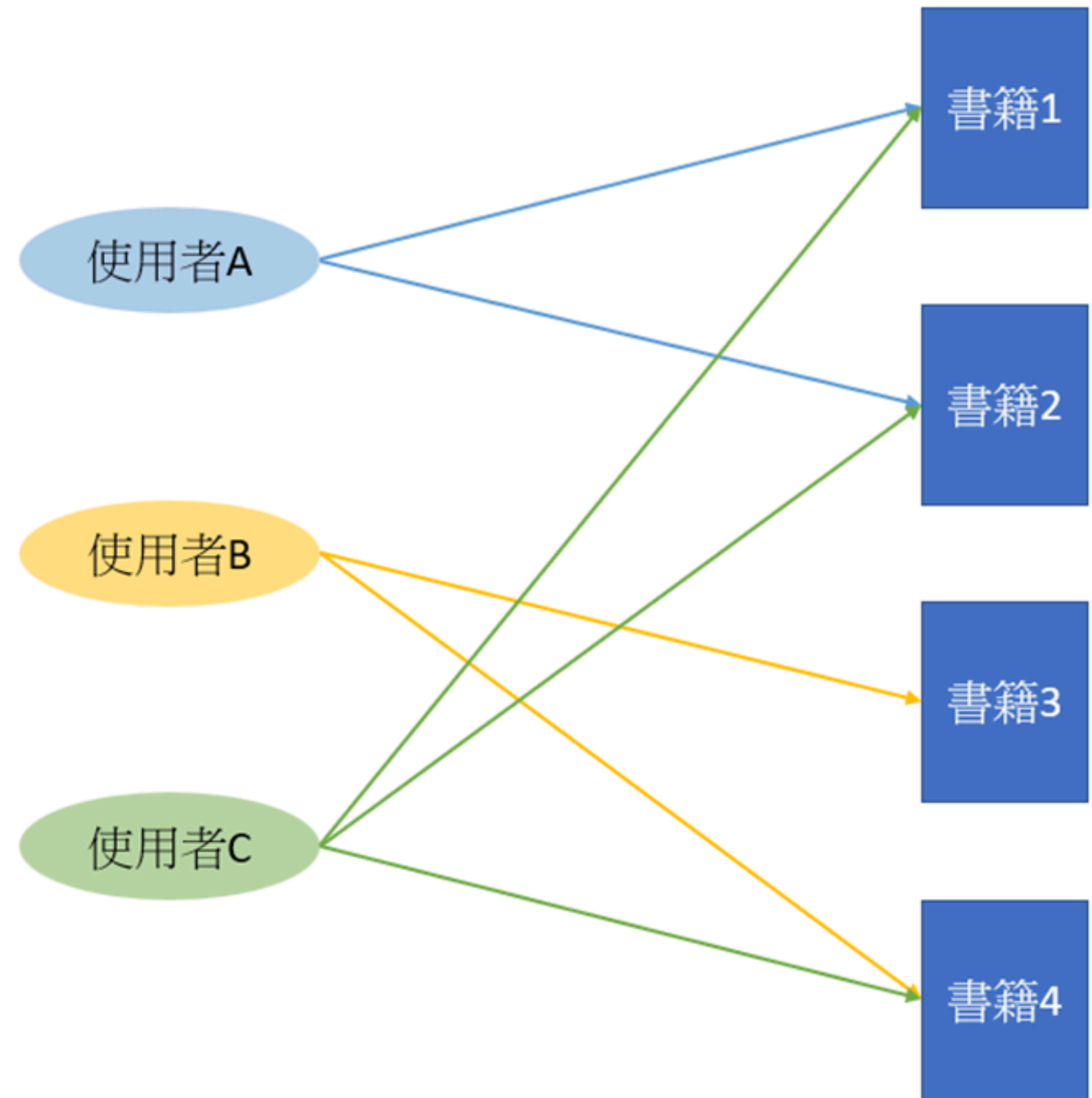
使用者偏好分析機制：協同過濾

目標:

分析目標使用者興趣，找到與他興趣相似的使用者並推薦其書單中目標使用者未收藏的書

舉例:

若要推薦一本**目標使用者A**未收藏的書給他，須先找到跟**A**興趣相似的使用者，如圖**A**及**C**的收藏重複率高，故從**C**已收藏但**A**未收藏的書中推薦書籍4給**A**



實體建置 搜尋演算法

Jaccard相似程度公式

作法1:

	書籍1	書籍2	書籍3	書籍4
使用者A	1	1	0	0
使用者B	0	0	1	1
使用者C	1	1	0	1

將圖轉換為喜好程度分析表，若一位使用者有收藏該本書則計為**1**，若無則為**0**

作法2:

目標使用者	使用者	相似程度
使用者A	使用者B	$(A \cap B) / (A \cup B) = 0/4 = 0$
使用者A	使用者C	$(A \cap C) / (A \cup C) = 2/3 \approx 0.67$

從兩人書單的聯集中，看交集的書所佔之比例

實體建置 搜尋演算法

Jaccard相似程度公式

作法3:

以二分法(收藏與沒收藏)來計算目標使用者對沒收藏的書及喜好程度

$$\sum_{i=1}^n \text{目標使用者與使用者}i\text{的相似程度} \\ \times \text{使用者}i\text{對書籍}k\text{的喜好程度}$$

缺點:

方式較像二分法，無法表達偏好程度

以書籍3、4為例，目標使用者A對他的喜好程度為:

$$\begin{aligned} & \text{A對書籍3的喜好程度：} \\ & = \text{A與B的相似程度} \times \text{B對書籍3的喜好程度} \\ & + \text{A與C的相似程度} \times \text{C對書籍3的喜好程度} \\ & = 0 \times 1 + 0.67 \times 0 = 0 \end{aligned}$$

$$\begin{aligned} & \text{A對書籍4的喜好程度：} \\ & = \text{A與B的相似程度} \times \text{B對書籍4的喜好程度} \\ & + \text{A與C的相似程度} \times \text{C對書籍4的喜好程度} \\ & = 0 \times 1 + 0.67 \times 1 = 0.67 \end{aligned}$$

實體建置 搜尋演算法

餘弦相似程度

解決方式:

把目標使用者A的觀看紀錄化成向量A，再對其他使用者B的觀看紀錄向量B做投影，投影長度越高表示相似程度越高。

公式:

喜好程度 = 使用者與目標使用者的相似程度
× 使用者觀看書籍之次數

$$\text{相似程度} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

實體建置 搜尋演算法

餘弦相似程度

作法1:

以目標使用者A、使用者B為例，用觀看次數作為喜好程度，將資料表read與自己做Natural JOIN並將喜好程度相乘，先計算出對同一本書籍時使用者的對應關係

書籍	使用者1	使用者2	喜好程度相乘
書籍1	A	A	AA
書籍1	A	B	AB
書籍1	B	A	BA
書籍1	B	B	BB

作法2:

用SUM計算 ΣA^2 、 ΣB^2 、 $A \times B$

作法3:

再將步驟(1)的表格與自己做第二次Natural JOIN，而需要將AA、AB、BB顯示在同一列已進行計算

P1.user1	P1.user2	pre	P2.user1	P2.user2	pre	P3.user1	P3.user2	pre
A	B	AB	A	A	AA	B	B	BB

實體建置 搜尋演算法

餘弦相似程度

作法4:

用公式「 $P1.pre / \sqrt{P2.pre * P3.pre}$ 」計算相似度投影

作法5:

用Natural JOIN合併read計算「A與B的相似度×B對書本的喜好程度」

作法6:

最後將預測出的喜好程度進行排列，推薦給使用者

實體建置 網頁

網頁使用Html搭配CSS，運用Flask框架，以及pymysql語法調用mysql資料庫，可直接在python 檔裡面使用mysql的指令

首頁:



測試

驗證1：所有書籍均需登錄於該系統內

事先將書籍資料整理成CSV檔匯入資料庫

```
mysql> USE mojoin;
Database changed
mysql> LOAD DATA LOCAL INFILE
-> 'D:/Users/peggy/Github/databaseproject-G9/Schema/csv/(table name).csv'
-> INTO TABLE `(table name)`
-> CHARACTER SET utf8mb4
-> FIELDS TERMINATED BY ',' -- 依據檔案的分隔符號
-> LINES TERMINATED BY '\n' -- 依據檔案每列的分隔方式
-> IGNORE 1 LINES;
```

再用pymysql語法導入mojoin資料庫

```
conn = pymysql.connect(
    host='localhost',
    user='root',
    password='12345678',
    database='mojoin',
    cursorclass=pymysql.cursors.DictCursor
)
```

執行結果(影片):

https://drive.google.com/file/d/16g_-BDxb69mNJb_FWWzcmkXYvZ6Mly6K/view?usp=sharing

測試

驗證2：可瀏覽網頁中所有書籍的資訊

程式:

```
#根據nIndex搜該書的書籍資訊(不含tags)
query = """
    SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
    FROM novel
    JOIN author ON novel.aIndex = author.aIndex
    JOIN publish ON novel.pIndex = publish.pIndex
    WHERE nIndex = %s
    """

cursor.execute(query, (nIndex,))
novel = cursor.fetchone()
if novel and 'Introduction' in novel:
    novel['Introduction'] = novel['Introduction'].replace('\r\n', '\n').replace('\r', '\n')

#根據nIndex搜該書的書籍資訊(tags)
query2 = """
    SELECT t.Tag
    FROM noveltag nt
    JOIN tag t ON nt.tIndex = t.tIndex
    WHERE nt.nIndex = %s AND nt.tIndex < 500
    ORDER BY t.tIndex;
    """

cursor.execute(query2, (nIndex,))
tags = cursor.fetchall()

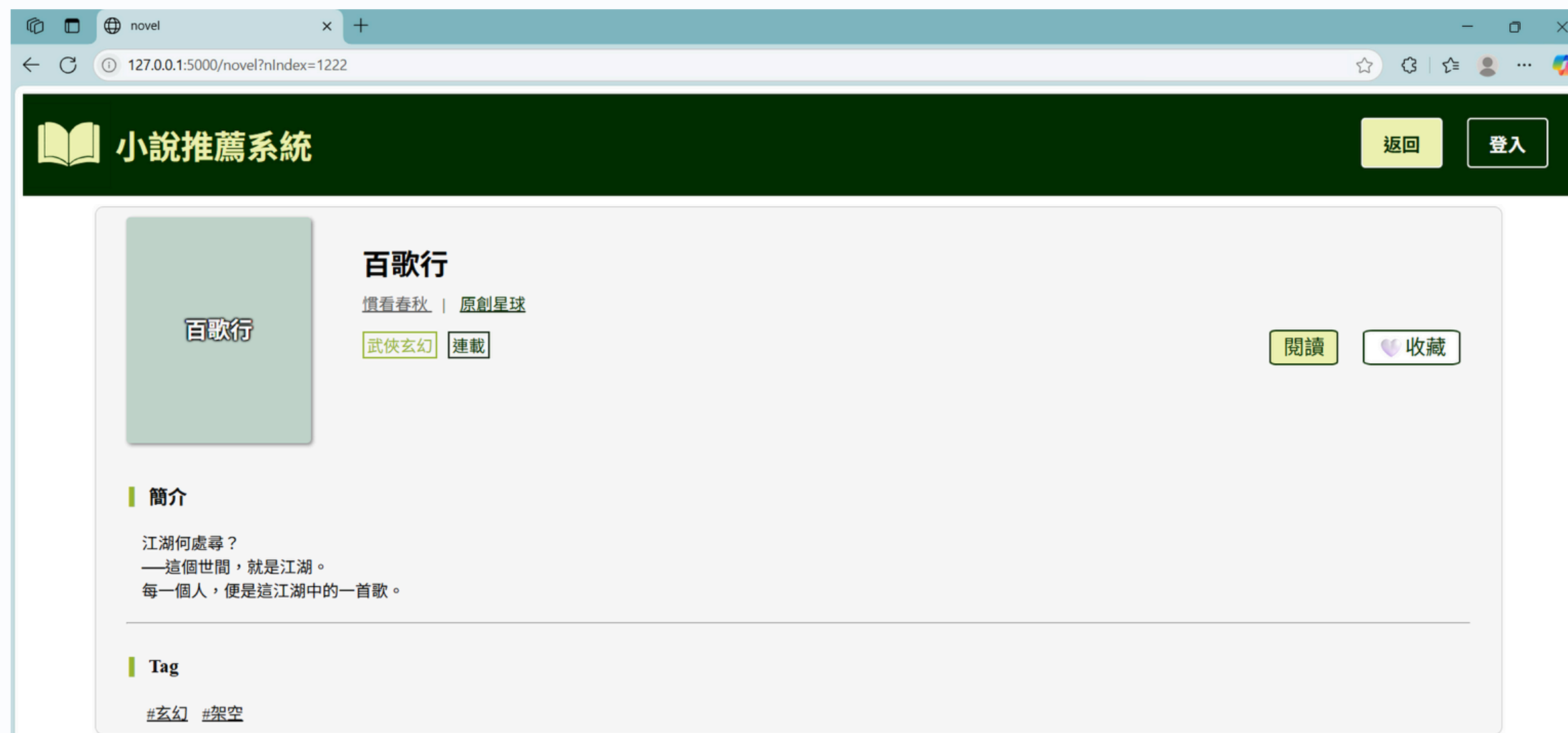
#搜使用者是否蒐藏該書籍(愛心之顯示)
cursor.execute("SELECT * FROM userlove WHERE Account = %s AND LoveRecord = %s", (username, nIndex))
is_loved = cursor.fetchone() is not None
```

測試

驗證2：可瀏覽網頁中所有書籍的資訊

執行結果：

每本小說的頁面都會顯示其書名、作者、出版社、類別、狀態、簡介與Tag，而閱讀按鍵會連結到小說網站。

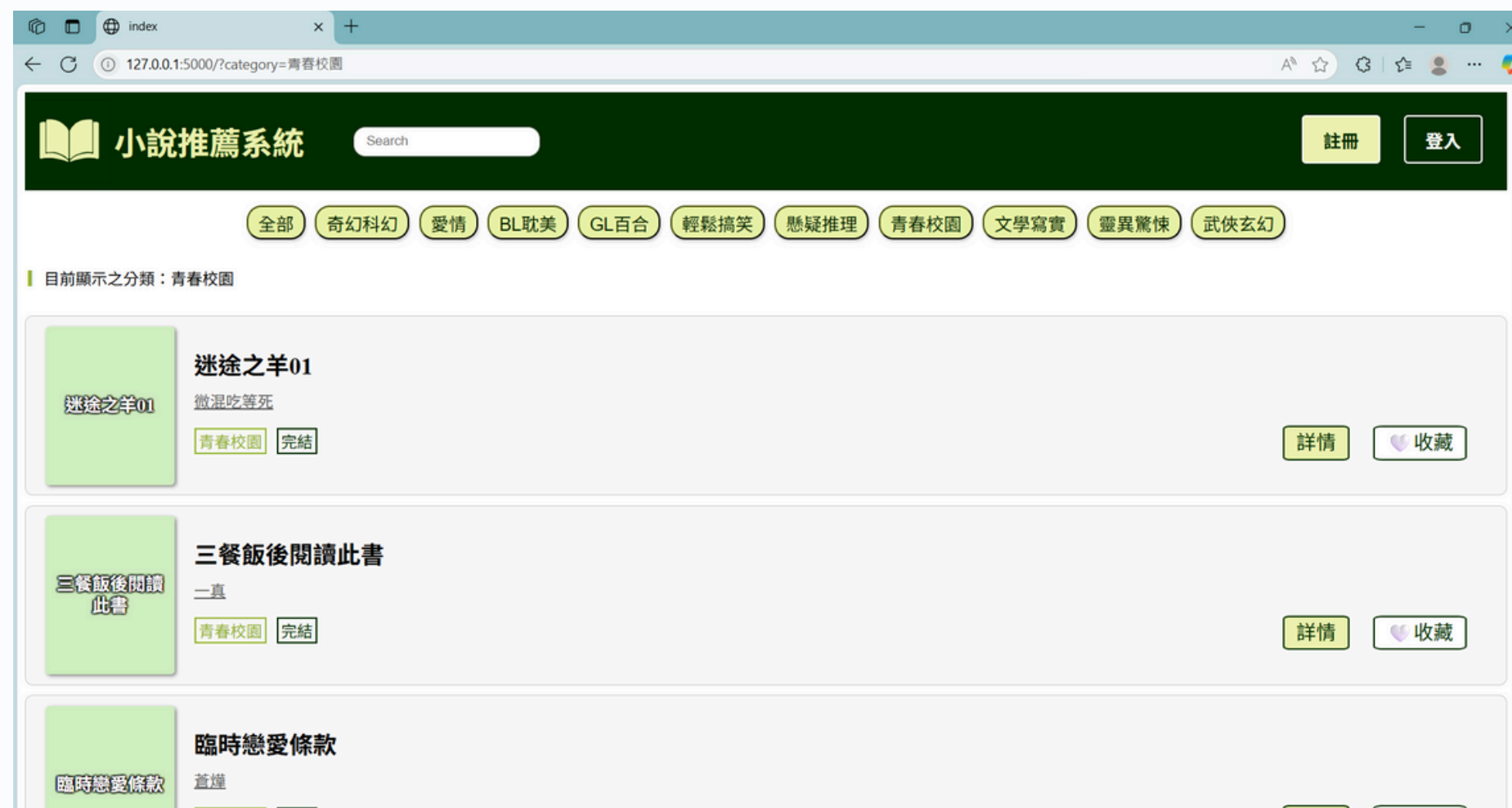


測試 類型篩選

驗證3：可經由類型、出版社、作者篩選書籍

```
elif category:
#搜尋同分類
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE novel.Category = %s
    """, (category,))
```

篩選類別：青春校園



測試 出版社篩選

驗證3：可經由類型、出版社、作者篩選書籍

```
elif publisher:
#搜尋同出版社
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE publish.Name = %s
    """, (publisher,))
```

篩選出版社：飛燕文創

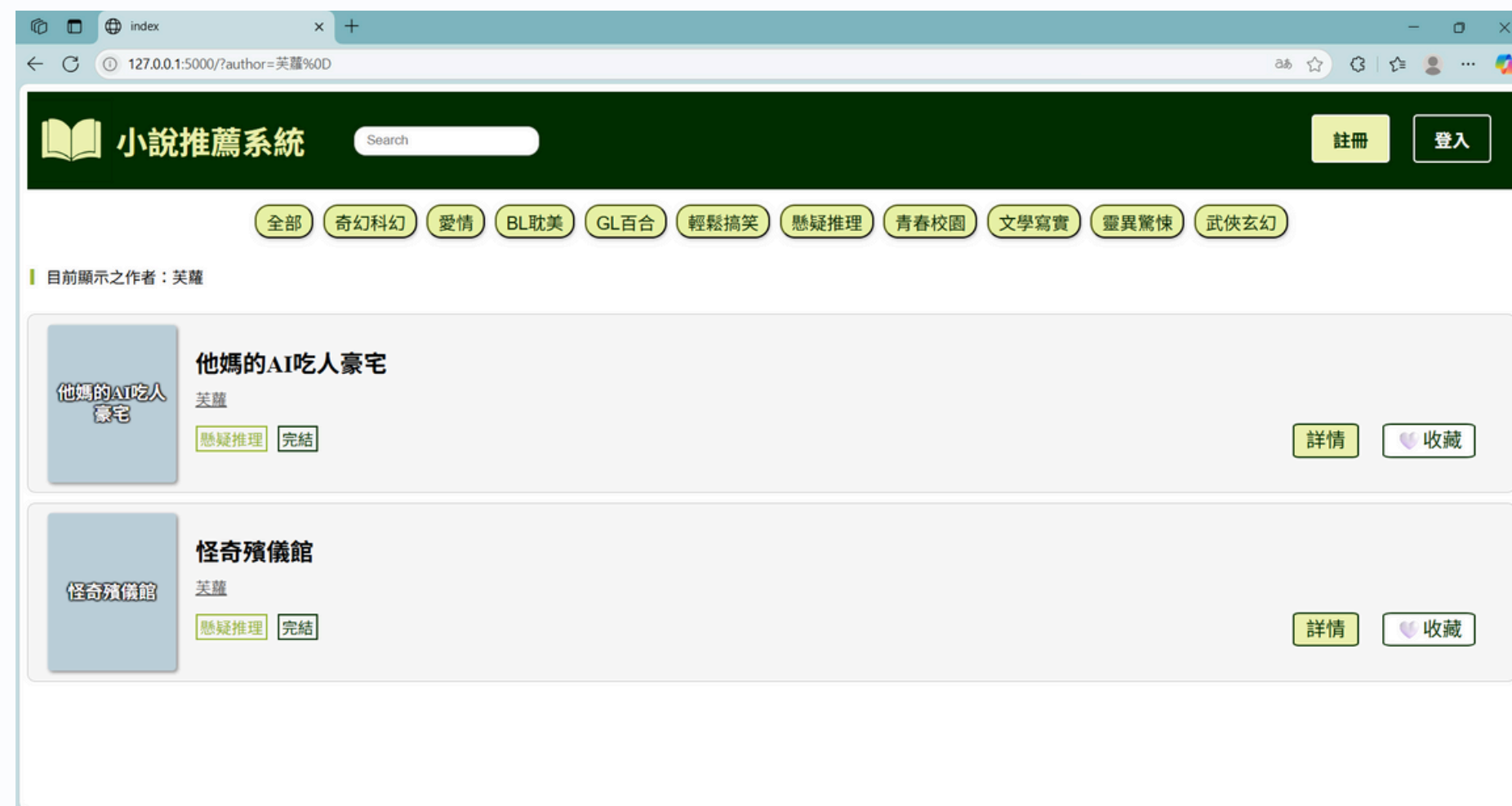


測試 作者篩選

驗證3：可經由類型、出版社、作者篩選書籍

```
elif author:
#搜尋同作者
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE author.Name = %s
    """, (author,))
```

篩選作者：芙蘿



測試 查找書籍

驗證4：可經由作者姓名、書籍名稱、出版社名稱或關鍵字

可依據使用者在Search欄位輸入之字詞查詢作者、書名、出版社等欄位包含該字詞的小說

```
if q:
#上方搜尋欄
    keyword = f"%{q}%"
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE novel.Title LIKE %s OR author.Name LIKE %s OR publish.Name LIKE %s
        """, (keyword, keyword, keyword))
    keyword = f"{q}"
```

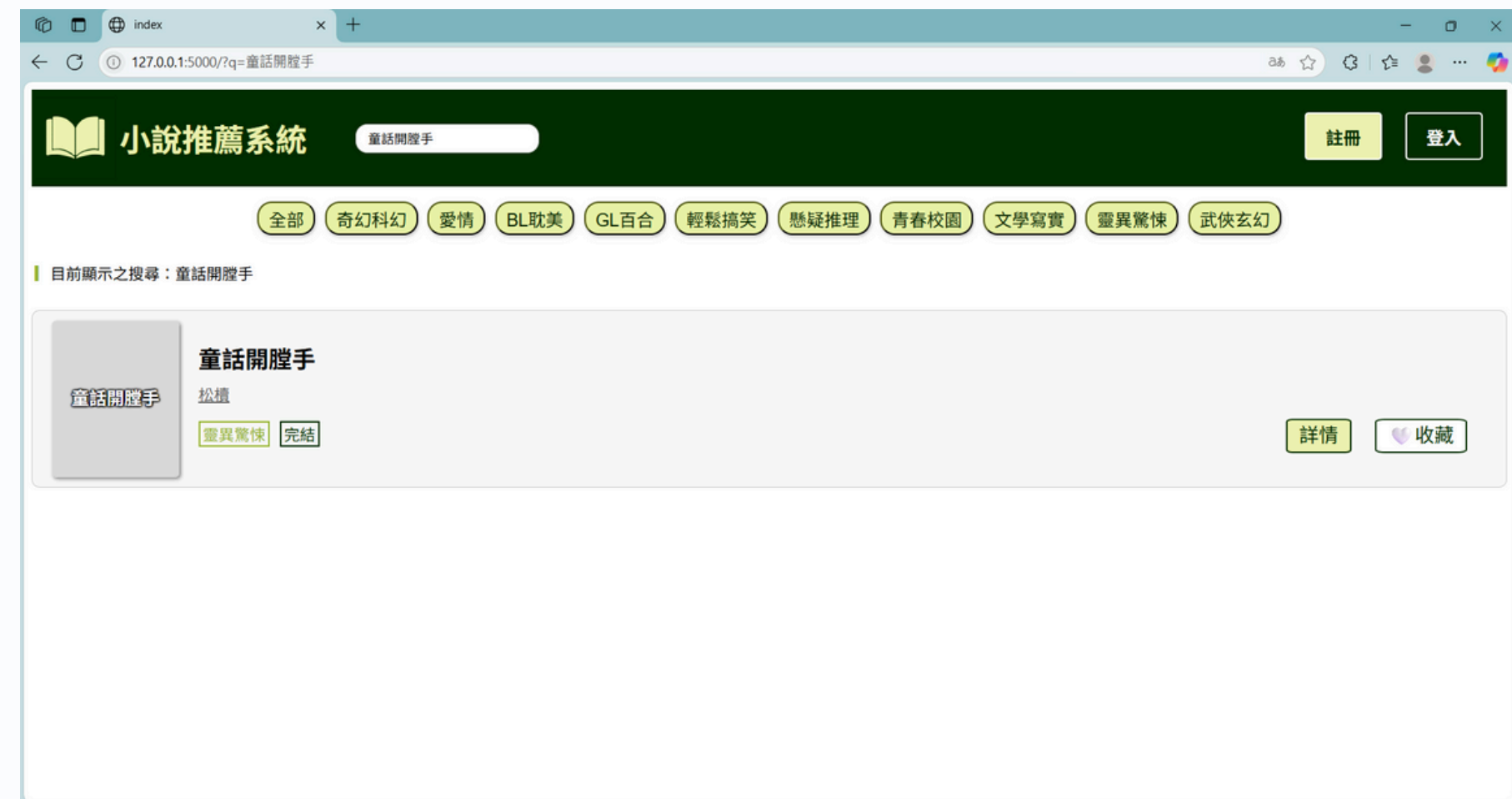
測試 查找書籍

驗證4：可經由作者姓名、書籍名稱、出版社名稱或關鍵字

搜尋作者姓名「吐維」



搜尋書籍名稱「童話開膛手」



測試 查找書籍

驗證4：可經由作者姓名、書籍名稱、出版社名稱或關鍵字

搜尋出版社「三日月書版」

搜尋關鍵字「非人類」



測試

驗證5：管理者可任意增加或删除書籍

以增加小說「無瞳之眼03」為例，因為該小說的作者、出版社與Tag已在author、publish和tag資料表中，故只需在novel和noveltag資料表中新增小說相關資訊。

```
INSERT INTO `mojoin`.`novel`  
(`nIndex`, `Title`, `Category`, `State`, `Love`, `Series`, `SeriesN`, `Link`, `Introduction`, `aIndex`, `pIndex`)  
VALUES ('1224', '無瞳之眼03', 'BL耽美', '完結', '130', '60', '3', 'https://mojoin.com/novels/3955',  
'提線的木偶抗拒著束縛 \r \r 正義小警察X神祕大作家—高潮迭起的懸疑篇章疑雲再起— \r \r 多年前的懸案再次浮出水面，徐  
'304', '2');  
INSERT INTO `mojoin`.`noveltag` (`nIndex`, `tIndex`) VALUES ('1224', '4');  
INSERT INTO `mojoin`.`noveltag` (`nIndex`, `tIndex`) VALUES ('1224', '151');  
INSERT INTO `mojoin`.`noveltag` (`nIndex`, `tIndex`) VALUES ('1224', '284');  
INSERT INTO `mojoin`.`noveltag` (`nIndex`, `tIndex`) VALUES ('1224', '288');
```

未增加小說「無瞳之眼03」前

1 • SELECT * FROM mojoin.novel WHERE mojoin.novel.Title='無瞳之眼03';

	nIndex	Title	Category	State	Love	Series	SeriesN	Warrant	Link	Introduct
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

增加後在資料庫可搜索到小說

1 • SELECT * FROM mojoin.novel WHERE mojoin.novel.Title='無瞳之眼03';

	nIndex	Title	Category	State	Love	Series	SeriesN	Warrant	Link
	1224	無瞳之眼03	BL耽美	完結	130	60	3	NULL	https://mojoin

測試

驗證5：管理者可任意增加或删除書籍

網頁上亦可看到該小說的相關資訊



測試 登入

驗證6：使用者須登入方可使用平台之推薦書籍功能



```
#搜尋是否有和使用者輸入之相同的帳密
cursor.execute("SELECT * FROM user WHERE Account=%s AND Password=%s", (username, password))
user = cursor.fetchone() #取出一筆資料
conn.close()

if user:
    #print('username=', username , 'password=', password , 'user=', user)
    #return render_template("login.html", error="登入成功")
    #已登入 儲存使用者名稱 返回index
    session['username'] = username
    return redirect(url_for('index'))
else:
    #print('username=', username , 'password=', password , 'user=', user)
    #登入失敗 顯示錯誤訊息
    return render_template("login.html", error="帳號或密碼錯誤")
```

login

127.0.0.1:5000/login



登入

帳號：

密碼：

登入

尚未有帳號，[點我註冊](#)

測試 登入

驗證6：使用者須登入方可使用平台之推薦書籍功能

因為帳號(Account)為主鍵，必須具有唯一性，因此在INSERT到user之前須先判斷有沒有與其重複之帳號名稱

```
# 確認密碼一致
if password != confirm_password:
    message = "密碼與確認密碼不一致，請重新輸入。"
    return render_template("register.html", message=message)

conn = pymysql.connect(
    host='localhost',
    user='root',
    password='J19940214k?',
    database='mojoin',
    cursorclass=pymysql.cursors.DictCursor
)
cursor = conn.cursor()

# 檢查帳號是否已存在
cursor.execute("SELECT * FROM user WHERE Account = %s", (username,))
existing_user = cursor.fetchone()

if existing_user:
    message = "帳號已存在，請使用其他帳號。"
else:
    # 將新的使用者帳密寫入資料庫
    cursor.execute("INSERT INTO user (Account, password) VALUES (%s, %s)", (username, password))
    conn.commit()
    message = "註冊成功！"
```



register

127.0.0.1:5000/register

註冊

帳號:

密碼:

確認密碼:

註冊

已有帳號, [點我登入](#)

測試 閱讀紀錄

驗證7：系統可記錄使用者閱讀紀錄、收藏紀錄

點擊閱讀按鈕後，History會加一，
記錄使用者閱讀過一次那本書

點擊小說「地球之眼」(nIndex: 30)前

```
# 先檢查是否有這筆資料，沒有就插入
cursor.execute("""
    SELECT * FROM `read` WHERE Account = %s AND nIndex = %s
""", (username, nid))
row = cursor.fetchone()

if row:
    # 若有紀錄則更新 History + 1
    cursor.execute("""
        UPDATE `read` SET History = History + 1 WHERE Account = %s AND nIndex = %s
        """, (username, nid))
else:
    # 若無則插入新的資料
    cursor.execute("""
        INSERT INTO `read` (Account, nIndex, History) VALUES (%s, %s, 1)
        """, (username, nid))
```

	Account	nIndex	History
▶	aaa	34	1
	aaa	110	1
	aaa	165	4
	aaa	333	2
	aaa	922	3
	aaa	1213	1

第一次

	Account	nIndex	History
▶	aaa	30	1

第二次

	Account	nIndex	History
▶	aaa	30	2

測試 收藏紀錄

驗證7：系統可記錄使用者閱讀紀錄、收藏紀錄

```
#搜使用者是否蒐藏該書籍(愛心之顯示)
cursor.execute("SELECT * FROM userlove WHERE Account = %s AND LoveRecord = %s", (username, nIndex))
exists = cursor.fetchone()

if exists:
    # 如果已經收藏 -> 取消收藏
    cursor.execute("DELETE FROM userlove WHERE Account = %s AND LoveRecord = %s", (username, nIndex))
else:
    # 沒有收藏 -> 加入收藏
    cursor.execute("INSERT INTO userlove (Account, LoveRecord) VALUES (%s, %s)", (username, nIndex))
```

當使用者點下收藏鍵:

1. 用**SELECT**判斷是否為收藏狀態
2. 已收藏: **DELETE**->取消收藏
未收藏: **INSERT** ->加入收藏

收藏小說「指數化愛情」(nIndex: 585)前

	Account	LoveRecord
▶	aaa	154
	aaa	922
	bbb	1111
	ccc	9
	ccc	165
	ccc	922
*	NULL	NULL

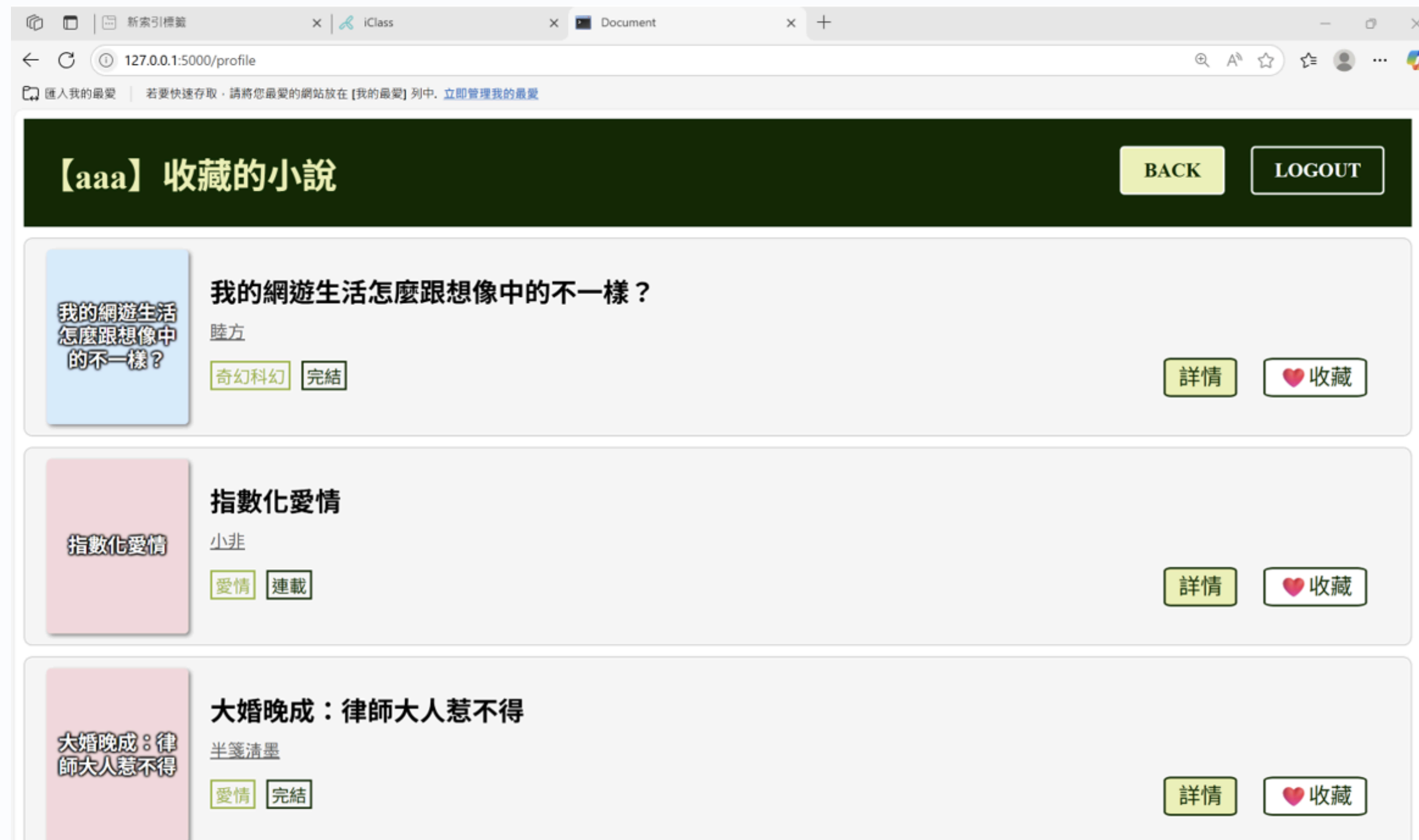
收藏小說「指數化愛情」(nIndex: 585)後

	Account	LoveRecord
▶	aaa	154
	aaa	585
	aaa	922
	bbb	1111
	ccc	9
	ccc	165
	ccc	922
*	NULL	NULL

測試 收藏紀錄

驗證7：系統可記錄使用者閱讀紀錄、收藏紀錄

是否收藏的結果也會同步顯示在網頁上



測試

驗證8：系統可綜合使用者的喜好推薦其可能感興趣之書籍

```
# 計算餘弦相似度
dot_product_matrix = np.dot(user_book_matrix.values, user_book_matrix.values.T)
norms = np.linalg.norm(user_book_matrix.values, axis=1)
norm_matrix = np.outer(norms, norms)
cosine_similarity_matrix = np.divide(
    dot_product_matrix,
    norm_matrix,
    out=np.zeros_like(dot_product_matrix),
    where=norm_matrix != 0
)
cosine_similarity_df = pd.DataFrame(
    cosine_similarity_matrix,
    index=user_book_matrix.index,
    columns=user_book_matrix.index
)

similarities = cosine_similarity_df.loc[user]
other_users = user_book_matrix.index.difference([user])
user_history = user_book_matrix.loc[user]
unread_books = user_history[user_history == 0].index

predicted_scores = {}
for book in unread_books:
    if book not in user_book_matrix.columns:
        continue
    scores = user_book_matrix.loc[other_users, book]
    sim_scores = similarities[other_users]
    mask = sim_scores > 0
    if mask.sum() > 0:
        weighted_sum = np.dot(scores[mask], sim_scores[mask])
        if weighted_sum > 0:
            predicted_scores[book] = weighted_sum
```

```
# 排序預測分數
sorted_books = sorted(predicted_scores.items(), key=lambda x: x[1], reverse=True)
top_books = set(id for id, _ in sorted_books[:top_n - random.randint(0, 5)])

# + 補推薦 (若不足 top_n)
if len(top_books) < top_n:
    left = top_n - len(top_books)
    read_books = set(user_history[user_history > 0].index)
    excluded_ids = read_books | top_books

    if excluded_ids:
        placeholders = ','.join(['%s'] * len(excluded_ids))
        query = f"""
            SELECT nIndex FROM novel
            WHERE nIndex NOT IN ({placeholders})
            ORDER BY RAND() LIMIT %s
        """
        params = list(excluded_ids) + [left * 5]
    else:
        query = "SELECT nIndex FROM novel ORDER BY RAND() LIMIT %s"
        params = [left * 5]

    cursor.execute(query, params)
    random_rows = cursor.fetchall()
    for row in random_rows:
        top_books.add(row['nIndex'])
        if len(top_books) >= top_n:
            break
```

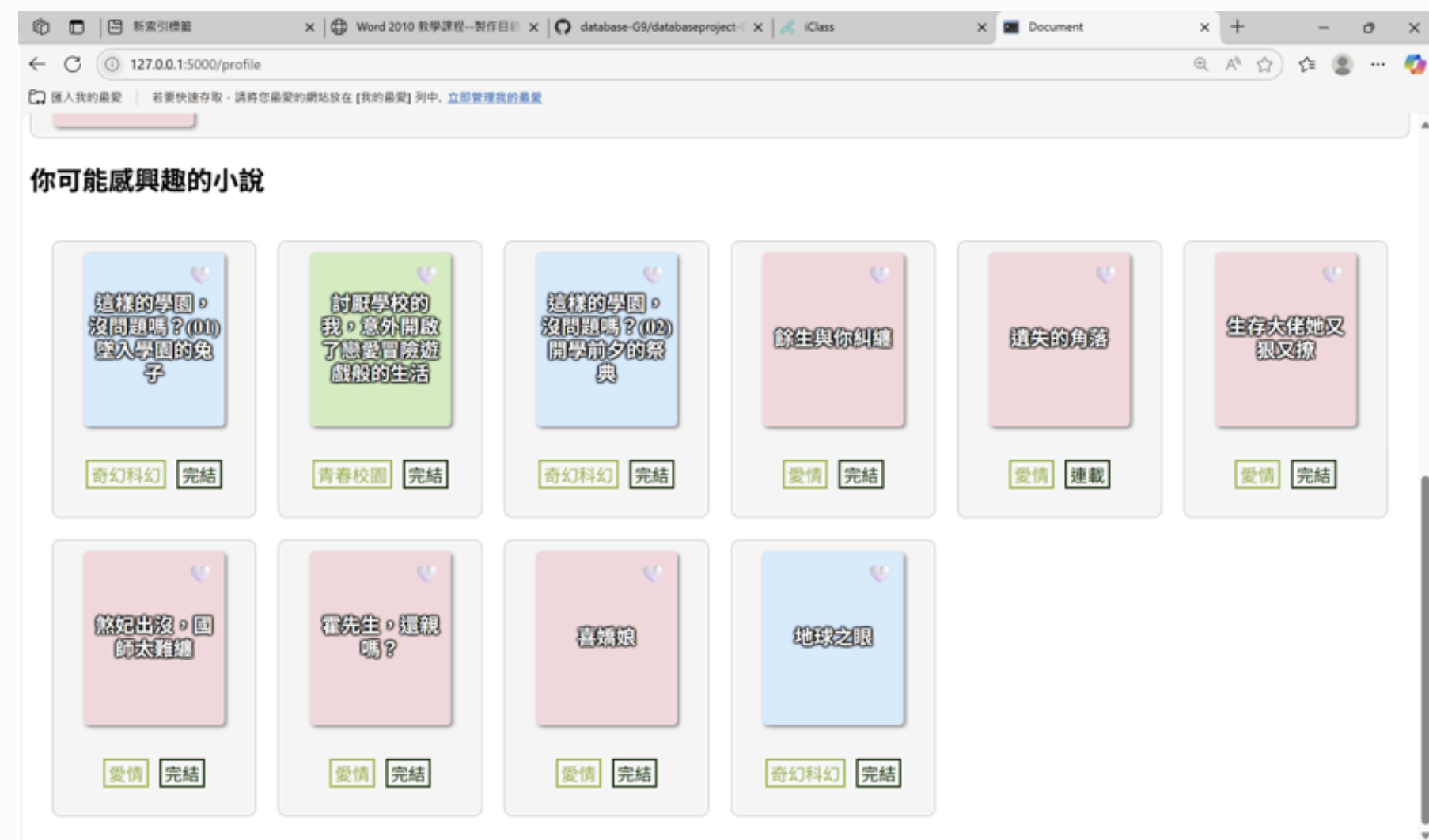
測試

驗證8：系統可綜合使用者的喜好推薦其可能感興趣之書籍

已ccc的閱讀紀錄為例

```
1 • USE mojoin;
2 • SELECT user.Account, mojoin.read.nIndex, novel.Title, novel.Category, mojoin.read.History
3   From user
4   LEFT JOIN mojoin.read
5   ON user.Account=mojoin.read.Account
6   LEFT JOIN novel
7   ON novel.nIndex=mojoin.read.nIndex
8   HAVING user.Account='ccc';
```

result Grid				
Account	nIndex	Title	Category	History
ccc	9	沒魔法資質的我在異世界靠科技成為救世主	奇幻科幻	3
ccc	110	第二世界幻想奇談	奇幻科幻	1
ccc	165	業火少女七奈	奇幻科幻	3
ccc	333	對手被隕石砸死難道是我的錯嗎？(02)冒險...	奇幻科幻	1
ccc	757	縮髮似君心	愛情	1
ccc	922	大婚晚成：律師大人惹不得	愛情	4
ccc	1111	女友是個想39歲安樂死的厭世傢伙	青春校園	1
ccc	1213	天地蒼年情之 繾綣雲(01)	武俠玄幻	2



測試

驗證9：系統可依據使用者當前查看的書推薦其可能感興趣之書籍

查找是否存在續集並推薦

```
# 1. 查出書的 Series 與 SeriesN
cursor.execute("""
    SELECT Series, SeriesN FROM novel WHERE nIndex = %s
""", (book_id,))
series_info = cursor.fetchone()
if not series_info:
    print(f"⚠ 找不到書籍 {book_id}。")
    return []

series_id = series_info['Series']
series_n = series_info['SeriesN']

# 2. 查找續集 (SeriesN > 現在的)
next_book_id = None
if series_id and series_n > 0:
    cursor.execute("""
        SELECT nIndex FROM novel
        WHERE Series = %s AND SeriesN > %s
        ORDER BY SeriesN ASC LIMIT 1
    """, (series_id, series_n))
    row = cursor.fetchone()
    if row:
        next_book_id = row['nIndex']
```

```
# 3. 查找前一集 (SeriesN < 現在的)
prev_book_id = None
if series_id and series_n > 0:
    cursor.execute("""
        SELECT nIndex FROM novel
        WHERE Series = %s AND SeriesN < %s
        ORDER BY SeriesN DESC LIMIT 1
    """, (series_id, series_n))
    row = cursor.fetchone()
    if row:
        prev_book_id = row['nIndex']

related_books = []
if next_book_id:
    related_books.append(next_book_id)
if prev_book_id:
    related_books.append(prev_book_id)
```

測試

驗證9：系統可依據使用者當前查看的書推薦其可能感興趣之書籍

依Tag推薦書籍

```
# 4. 查 tag
cursor.execute("""
    SELECT tIndex FROM noveltag WHERE nIndex = %s
""", (book_id,))
tags = [row['tIndex'] for row in cursor.fetchall()]
if not tags:
    print(f"⚠ 書籍 {book_id} 沒有 tag 資料。")
    return [get_book(cursor, bid) for bid in related_books]
```

```
# 5. 所有書 tag 資料
cursor.execute("SELECT nIndex, tIndex FROM noveltag")
all_tag_data = cursor.fetchall()

tag_to_books = {}
for row in all_tag_data:
    if int(row['nIndex']) == int(book_id):
        continue
    tag_to_books.setdefault(row['nIndex'], set()).add(row['tIndex'])
```

```
# 6. tag 相似度分數
predicted_scores = {}
for other_book_id, tag_set in tag_to_books.items():
    if int(other_book_id) == int(book_id) or other_book_id in related_books:
        continue
    overlap = len(set(tags) & tag_set)
    if overlap > 0:
        predicted_scores[other_book_id] = overlap

# 移除本書，避免推薦自己
predicted_scores.pop(book_id, None)

sorted_books = sorted(predicted_scores.items(), key=lambda x: x[1], reverse=True)
top_books = []
for bid, _ in sorted_books:
    if bid != book_id and bid not in top_books and bid not in related_books:
        top_books.append(bid)
    if len(top_books) >= (top_n - len(related_books)):
        break
```

測試

驗證9：系統可依據使用者當前查看的書推薦其可能感興趣之書籍

隨機補滿書籍

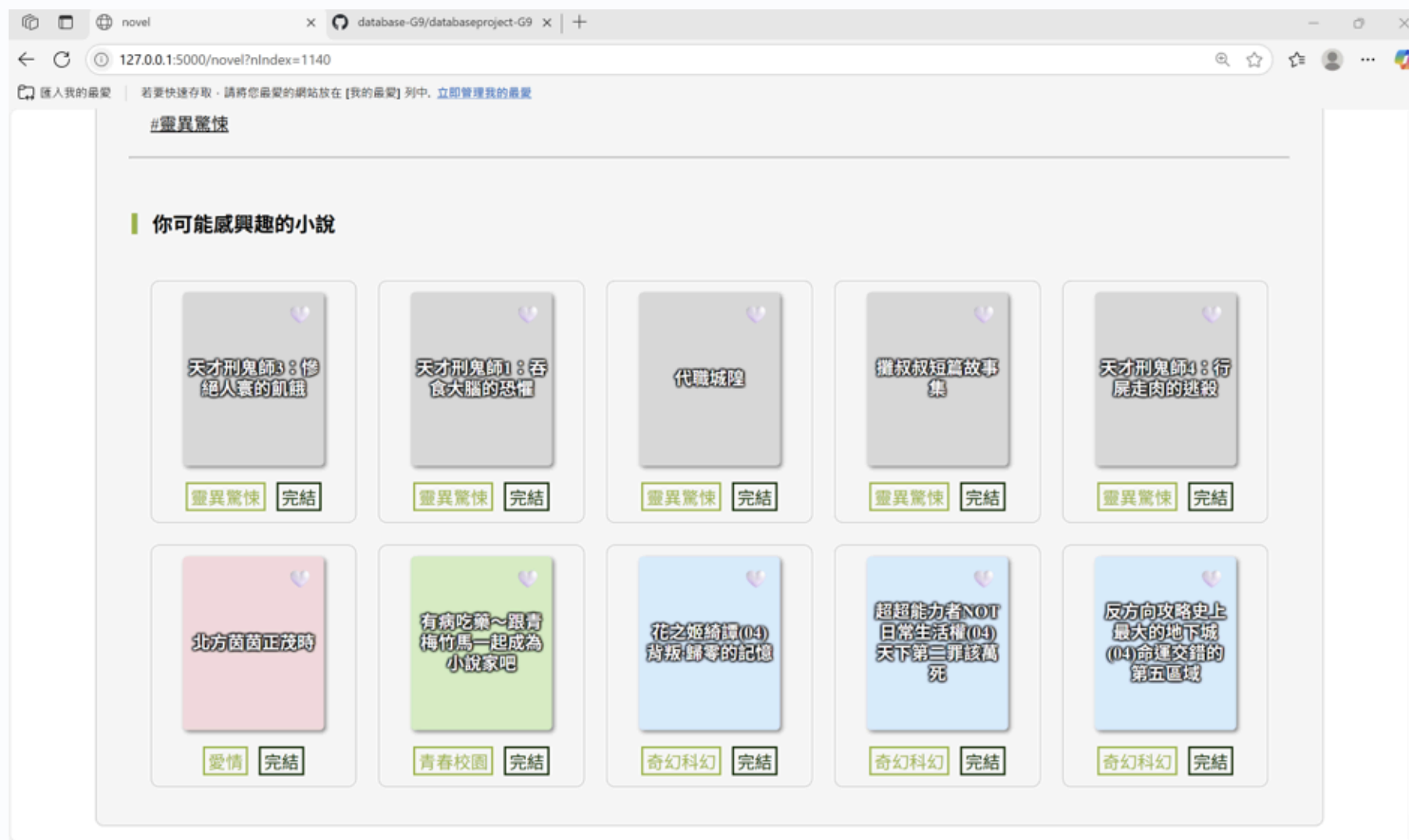
```
# 7. 補齊推薦 (隨機)
if len(top_books) + len(related_books) < top_n:
    needed = top_n - len(top_books) - len(related_books)
    excluded = top_books + related_books + [book_id]
    placeholders = ','.join(['%s'] * len(excluded))
    query = f"""
        SELECT nIndex FROM novel
        WHERE nIndex NOT IN ({placeholders})
        ORDER BY RAND() LIMIT %s
    """
    params = excluded + [needed * 3]
    cursor.execute(query, params)
    for row in cursor.fetchall():
        bid = row['nIndex']
        if bid != book_id and bid not in top_books and bid not in related_books:
            top_books.append(bid)
            if len(top_books) + len(related_books) >= top_n:
                break

final_books = [bid for bid in (related_books + top_books) if bid != book_id]
```

測試

驗證9：系統可依據使用者當前查看的書推薦其可能感興趣之書籍

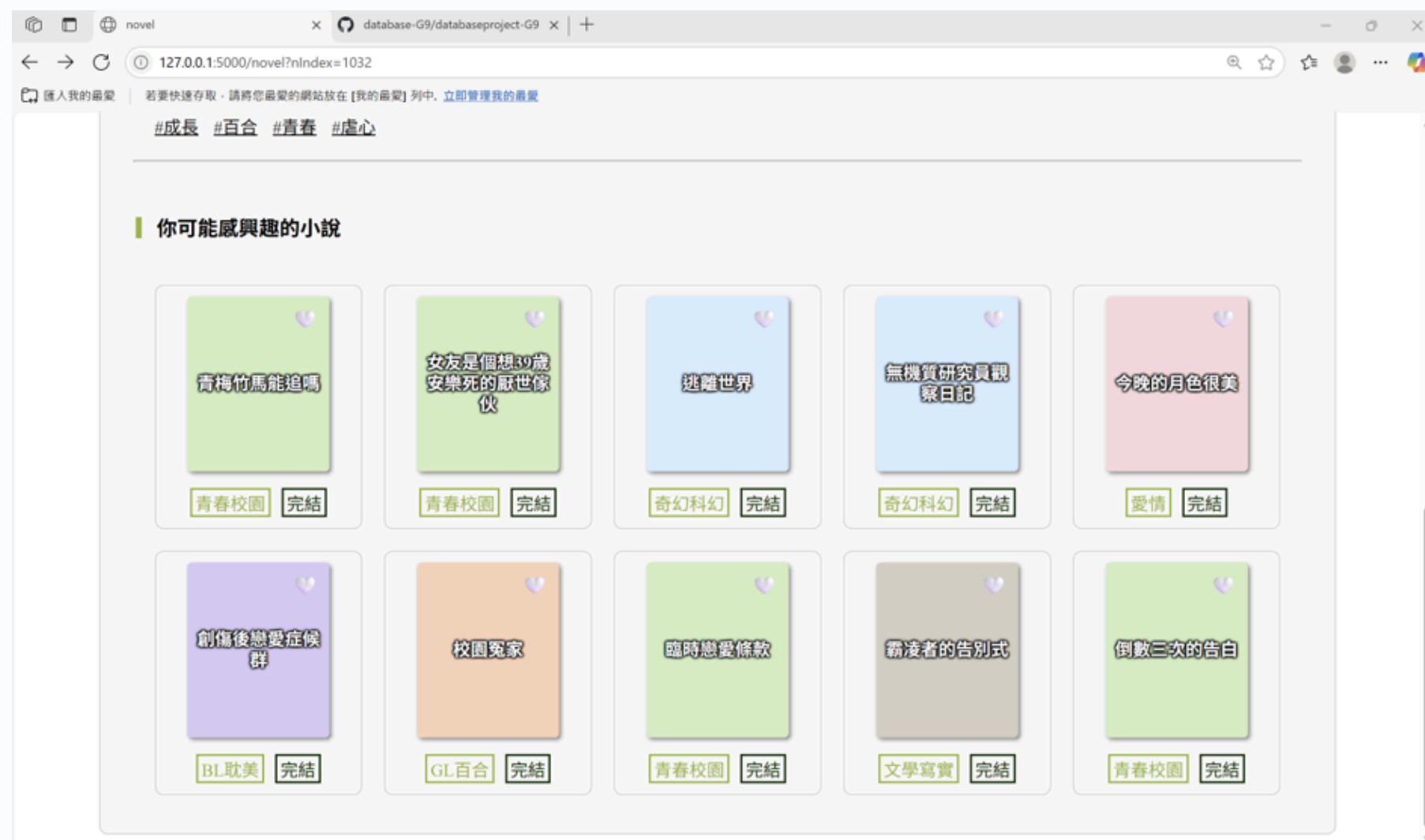
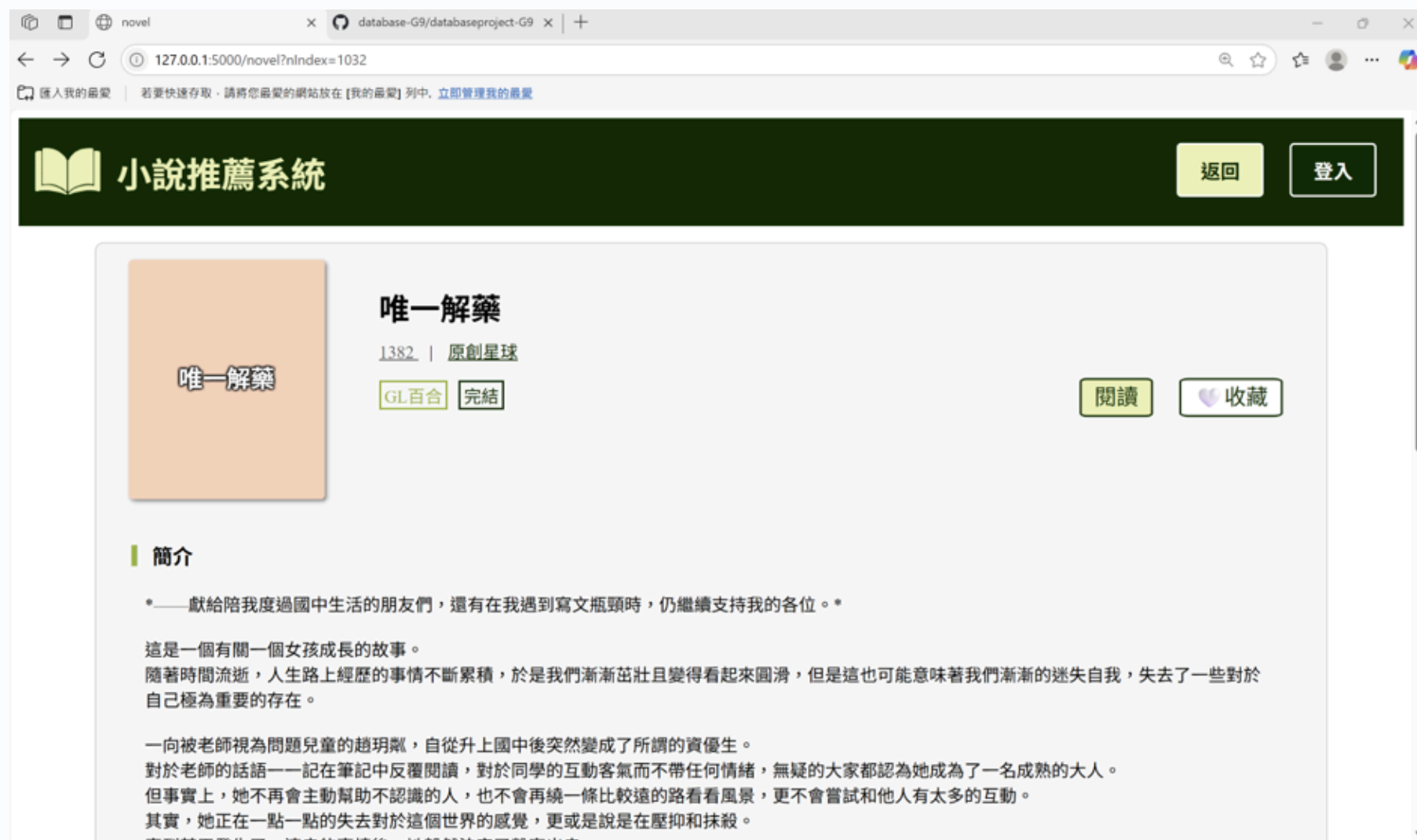
系列中的小說



測試

驗證9：系統可依據使用者當前查看的書推薦其可能感興趣之書籍

並非系列的小說



測試 額外功能

可依據使用者選擇之狀態篩選小說

使用者可點選小說狀態之按鍵，篩選同狀態小說

篩選狀態：連載

```
elif state:
#搜尋同狀態
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE State = %s
    """, (state,))
```



測試 額外功能

可依據使用者選擇之Tag篩選小說

可依據使用者點擊之Tag連結篩選同Tag之小說

選Tag：平行時空

```
elif tag:
    #搜尋同狀態
    cursor.execute("""
        SELECT novel.*, author.Name AS AuthorName, publish.Name AS PublishName
        FROM novel
        JOIN noveltag nt ON nt.nIndex = novel.nIndex
        JOIN tag t ON t.tIndex = nt.tIndex
        JOIN author ON novel.aIndex = author.aIndex
        JOIN publish ON novel.pIndex = publish.pIndex
        WHERE t.tIndex = nt.tIndex AND nt.nIndex = novel.nIndex AND t.Tag = %s
    """, (tag,))
```



測試



影片連結:

<https://drive.google.com/file/d/1Y-YDbeEbeXWEdJQQ3umhC68Bi9VLTmG8/view?usp=drivesdk>



結論

- 目前方式

1. 資料庫：MySQL
2. 推薦演算法：餘弦相似度
3. 網頁：Html搭配Flask架構，使用CSS排版

- 功能

1. 管理者：操作資料庫刪減、新增或更新資料
2. 使用者：使用網頁新增使用者、收藏與查找書籍

- 未來展望

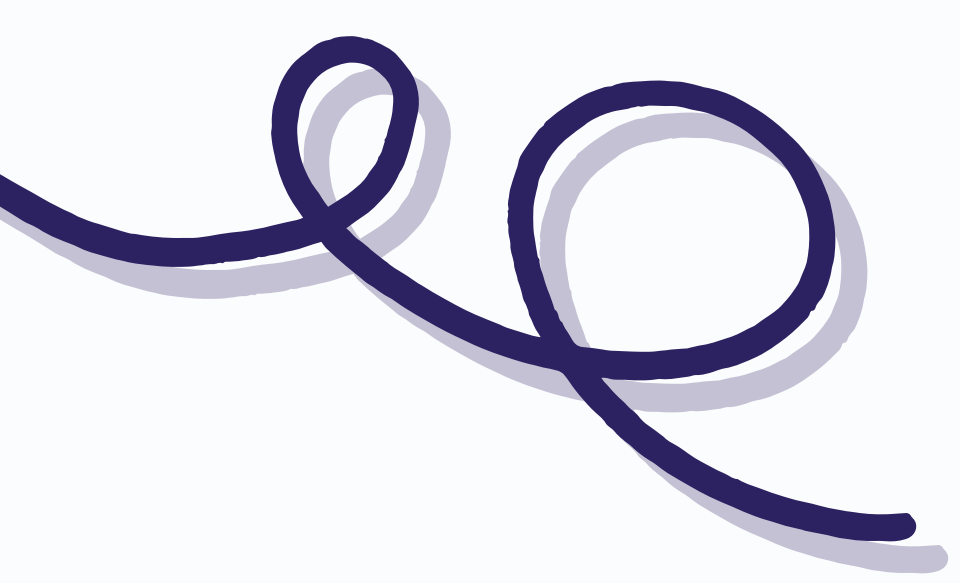
1. 在網頁使用人數較低時改成運用另一套演算法，等到使用者足夠時再改用餘弦相似度計算，以提升推薦準確率

- 總結

1. 此系統雖仍有進步空間，但基本搜尋功能與推薦以相當完善

工作分配

	張子頡	林鈺丞	呂秉諺	劉奕霈	張庭瑄	金庭聿
資料蒐集	V	V	V	V	V	V
資料庫	V			V		
網頁				V	V	V
演算法		V	V			
書面報告			V	V		
簡報						V
影片	V					
口頭報告	V	V	V	V	V	V



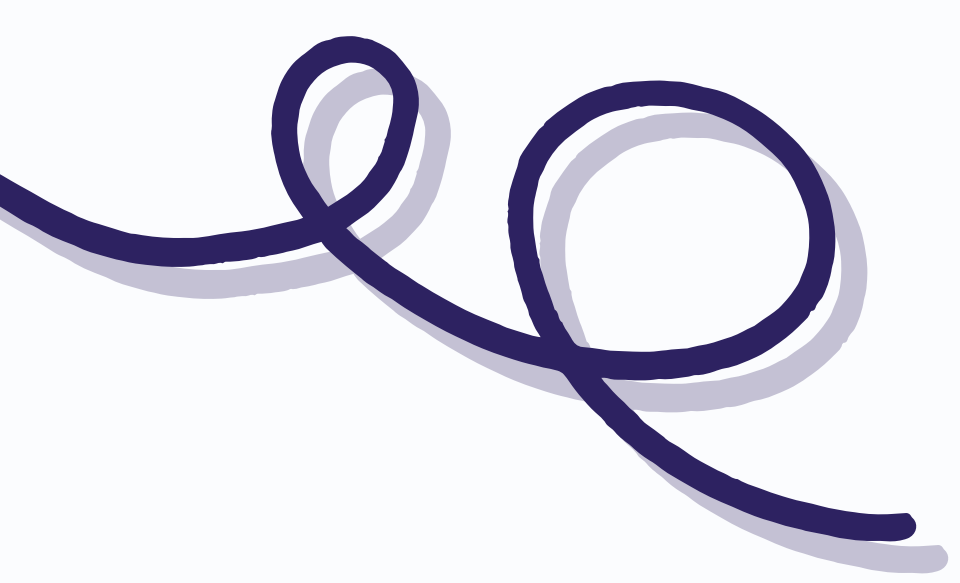
參考資料



資料來源：<https://mojoin.com/novels>

演算法結構：<https://zhuanlan.zhihu.com/p/489861857>





報告結束
謝謝大家

